

T2J-Bench: A Bug-Fix Benchmark for PyTorch-to-JAX Code Translation

Anonymous ACL submission

Abstract

Large Language Models (LLMs) have shown promising results for programming language translation, yet their reliability remains limited for domain-specific languages such as for PyTorch to JAX translation. In this work, we introduce **T2J-Bench**, an expanded benchmark and optimization framework for studying and improving LLM-generated PyTorch-to-JAX translations. We define five correctness criteria for translated JAX programs: compilation correctness, test-case correctness, loss-function correctness, readability correctness, and runtime correctness. Using these criteria, two senior software developers manually evaluate LLM-generated JAX code and perform multiple rounds of debugging until the translated programs satisfy the required correctness conditions. This process yields a dataset of **600** PyTorch-to-JAX translation bugs and corresponding human-written fixes, providing a fine-grained resource for analyzing common failure modes in LLM-based domain-specific code translation. We further demonstrate the utility of T2J-Bench through two LLM optimization strategies: in-context learning and fine-tuning. The resulting optimized models are evaluated on both JAX bug fixing and end-to-end PyTorch-to-JAX code translation. Experimental results show that models enhanced with T2J-Bench produce up to **60% more correct JAX code snippets** than the original LLM.

1 Introduction

Large Language Models (LLMs) have demonstrated strong potential for automated code translation across general-purpose programming languages. Recent benchmarks and empirical studies show that LLMs can translate code among languages such as C, C++, Java, Python, and Go, supporting software migration, cross-language interoperability, and modernization of legacy systems (Lu et al., 2021; Pan et al., 2024). These advances suggest that LLMs can capture syntactic

patterns, library usage, and common programming idioms across languages. However, even in general-purpose settings, code translation remains challenging because a translated program must preserve the behavior of the source program rather than merely resemble it textually. This challenge becomes even more difficult when translation involves specialized programming ecosystems whose semantics depend on domain-specific libraries, execution models, and optimization behavior.

High-performance computing (HPC) code translation introduces additional requirements beyond conventional source-to-source translation. In many general-purpose translation tasks, metrics such as BLEU, CodeBLEU, or syntax similarity are often used to estimate translation quality (Ren et al., 2020; Lu et al., 2021). However, for HPC programs, textual similarity alone is insufficient because a translated program must also compile, pass correctness tests, preserve numerical behavior, and maintain acceptable execution efficiency. Recent studies on LLM-based code translation emphasize that execution efficiency is a critical but often overlooked dimension of translation quality, especially when translated programs may be used in performance-sensitive workloads (Gong et al., 2025). Therefore, evaluating HPC code translation requires a richer notion of correctness that includes test-case correctness, runtime behavior, and domain-specific implementation constraints.

PyTorch-to-JAX translation can be viewed as a subproblem of HPC and machine-learning code translation. Both PyTorch and JAX are widely used for numerical computing and deep learning, but they differ substantially in execution semantics, automatic differentiation, state management, random-number generation, and compilation workflows. JAX provides a NumPy-like programming interface together with composable transformations such as automatic differentiation, vectorization, and just-in-time compilation (Bradbury et al., 2018). These

features make JAX attractive for high-performance machine-learning workloads, scientific computing, and accelerator-backed training or inference. However, these same features also make PyTorch-to-JAX translation difficult: a correct translation must preserve tensor shapes, model definitions, training loops, loss functions, gradient computations, and random-key handling while also following JAX-specific programming conventions.

Prior work on T2J made an important first step toward improving PyTorch-to-JAX translation by using human-fixed LLM translation bugs as contextual guidance for future translations (Authors, 2025). Nevertheless, the original T2J formulation has several limitations. First, it does not provide a sufficiently explicit definition of program equivalence between the input PyTorch program and the translated JAX program. In practice, equivalence checking is mainly performed through human inspection and by running limited test cases, often only a single test case per input program. As a result, the evaluation may miss deeper behavioral mismatches, especially errors involving loss functions, training dynamics, numerical edge cases, or incomplete coverage of input conditions. Second, the scale of the original T2J dataset is limited: it contains roughly 160 bug–solution pairs, which restricts its ability to characterize the broader space of PyTorch-to-JAX translation failures and limits its usefulness for data-driven LLM optimization.

To address these limitations, we introduce **T2J-Bench**, a larger benchmark and optimization resource for studying LLM-generated bugs in PyTorch-to-JAX translation. T2J-Bench extends the original T2J direction by defining a clearer correctness framework for translated JAX code and by collecting a substantially larger set of human-validated bug–fix examples. Instead of treating correctness as a single informal judgment, we evaluate translated programs according to multiple dimensions, including compilation correctness, test-case correctness, loss-function correctness, readability correctness, and runtime correctness. This design enables more systematic analysis of translation failures and provides a stronger foundation for improving LLMs through in-context learning and fine-tuning. Replication package is available at this site¹.

The main contributions of this work are summarized as follows:

1. We define a multi-dimensional correctness framework for PyTorch-to-JAX translation, covering compilation correctness, test-case correctness, loss-function correctness, and readability correctness.
2. We construct **T2J-Bench**, a developer-curated dataset of 600 PyTorch-to-JAX translation bugs and corresponding fixes, substantially expanding the scale of the original T2J dataset.
3. We evaluate the usefulness of T2J-Bench for improving LLM-based translation through two optimization strategies: in-context learning and fine-tuning.
4. We show that models enhanced with T2J-Bench can produce up to 60% more correct JAX code snippets than their original counterparts, demonstrating the value of human-curated bug–fix data for domain-specific code translation.

2 Program Equivalent Checking

Given an input PyTorch program P and its translated JAX program J , our goal is to determine whether J is semantically equivalent to P . We define program equivalence as a multi-dimensional correctness property. A translated JAX program is considered correct only when it satisfies five criteria: Compilation Correctness (CC), Testcase Correctness (TC), Loss Message Correctness (LMC), Readability Correctness (RC), and Running Time Correctness (RTC).

Formally, let $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ denote a set of generated test cases for the input PyTorch program P . We define the equivalence decision function as:

$$\text{Equiv}(P, J, \mathcal{T}) = \mathbb{I} \left[\begin{aligned} &CC(P, J) \wedge TC(P, J, \mathcal{T}) \\ &\wedge LMC(P, J, \mathcal{T}) \wedge RC(P, J) \\ &\wedge RTC(P, J, \mathcal{T}) \end{aligned} \right]. \quad (1)$$

where $\mathbb{I}[\cdot]$ is an indicator function that returns 1 if all conditions are satisfied and 0 otherwise. Thus, a translated JAX program is accepted as correct only if it passes every criterion.

Compilation Correctness (CC). Compilation Correctness checks whether the translated JAX code is syntactically valid, importable, and executable without compilation-time or initialization-time errors. This criterion captures basic failures

¹<https://tinyurl.com/4s3w8jun>

such as invalid Python syntax, missing imports, undefined variables, incorrect use of JAX or Flax APIs, invalid function signatures, and shape errors that prevent the program from starting. We define:

$$CC(J) = \begin{cases} 1, & \text{if } J \text{ can be compiled,} \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Testcase Correctness (TC). Testcase Correctness evaluates whether the translated JAX program produces outputs consistent with the input PyTorch program across a set of generated test cases. For each test case $t_i \in \mathcal{T}$, we execute both P and J under aligned inputs and comparable random seeds. Let $O_P(t_i)$ be the output of the PyTorch program and $O_J(t_i)$ be the output of the translated JAX program. Since machine learning programs frequently involve floating-point computation, we use approximate numerical equality rather than exact equality:

$$\mathbb{I} \left[\frac{\|O_P(t_i) - O_J(t_i)\|_2}{\max(\|O_P(t_i)\|_2, \epsilon)} \leq \delta \right], \quad (3)$$

where δ is a numerical tolerance and ϵ is a small constant used to avoid division by zero. We then define testcase correctness as:

$$TC(P, J, \mathcal{T}) = \mathbb{I} \left[\frac{1}{|\mathcal{T}|} \sum_{t_i \in \mathcal{T}} \text{match}(O_P(t_i), O_J(t_i)) = 1 \right]. \quad (4)$$

Loss Message Correctness (LMC). Loss Message Correctness checks whether the translated JAX program preserves the loss behavior of the original PyTorch program. This criterion is important because many machine learning programs do not only produce a final prediction; they also define a loss function, compute gradients, and update model parameters. A translation may pass simple output tests while still implementing an incorrect loss function or gradient computation. Let $\ell_P^{(k)}$ and $\ell_J^{(k)}$ denote the loss values produced by the PyTorch and JAX programs at training step or epoch k , respectively. We define loss consistency as:

$$\text{LossSim}(P, J) = \frac{1}{K} \sum_{k=1}^K \mathbb{I} \left[\frac{|\ell_P^{(k)} - \ell_J^{(k)}|}{\max(|\ell_P^{(k)}|, \epsilon)} \leq \tau \right], \quad (5)$$

where K is the number of observed training steps or epochs and τ is the loss tolerance. The LMC criterion is then:

$$LMC(P, J, \mathcal{T}) = \mathbb{I} [\text{LossSim}(P, J) = 1]. \quad (6)$$

Readability Correctness (RC). Readability Correctness evaluates whether the translated JAX code is understandable, maintainable, and faithful to the structure of the input PyTorch implementation. This criterion is included because translated code can sometimes pass tests while being overly complex, incomplete, non-idiomatic, or difficult to modify. Verifiers inspect whether the JAX code uses appropriate abstractions, meaningful function boundaries, clear parameter passing, and idiomatic JAX constructs. They also check whether the implementation contains placeholder functions, unused variables, dead code, hard-coded test outputs, or misleading logic. We define:

$$RC(P, J) = \begin{cases} 1, & \text{if } \forall s \in \mathcal{S}(P), \exists s' \in \mathcal{S}(J) \\ & \text{such that } \text{Match}(s, s') = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

Here, $\mathcal{S}(P)$ denotes the set of important statements and functions in the input PyTorch program P , while $\mathcal{S}(J)$ denotes the set of corresponding statements and functions implemented in the translated JAX program J . The function $\text{Match}(s, s')$ returns 1 if the JAX statement or function s' correctly implements the behavior of the PyTorch statement or function s , and returns 0 otherwise. In our evaluation process, this readability and implementation-coverage judgment is first performed by junior developers and then verified by senior developers. This readability check can also be supported by an LLM-as-a-judge, which provides an additional assessment of whether the translated JAX code faithfully implements the PyTorch logic.

Running Time Correctness (RTC). Running Time Correctness checks whether the translated JAX program has comparable runtime behavior to the original PyTorch program. Let $R_P(t_i)$ and $R_J(t_i)$ denote the execution time of the PyTorch and JAX programs on test case t_i . We define the average runtime ratio as:

$$\rho(P, J, \mathcal{T}) = \frac{1}{|\mathcal{T}|} \sum_{t_i \in \mathcal{T}} \frac{R_J(t_i)}{\max(R_P(t_i), \epsilon)}. \quad (8)$$

The translated JAX program satisfies RTC if this ratio is below a predefined threshold γ :

$$RTC(P, J, \mathcal{T}) = \mathbb{I} [\rho(P, J, \mathcal{T}) \leq \gamma]. \quad (9)$$

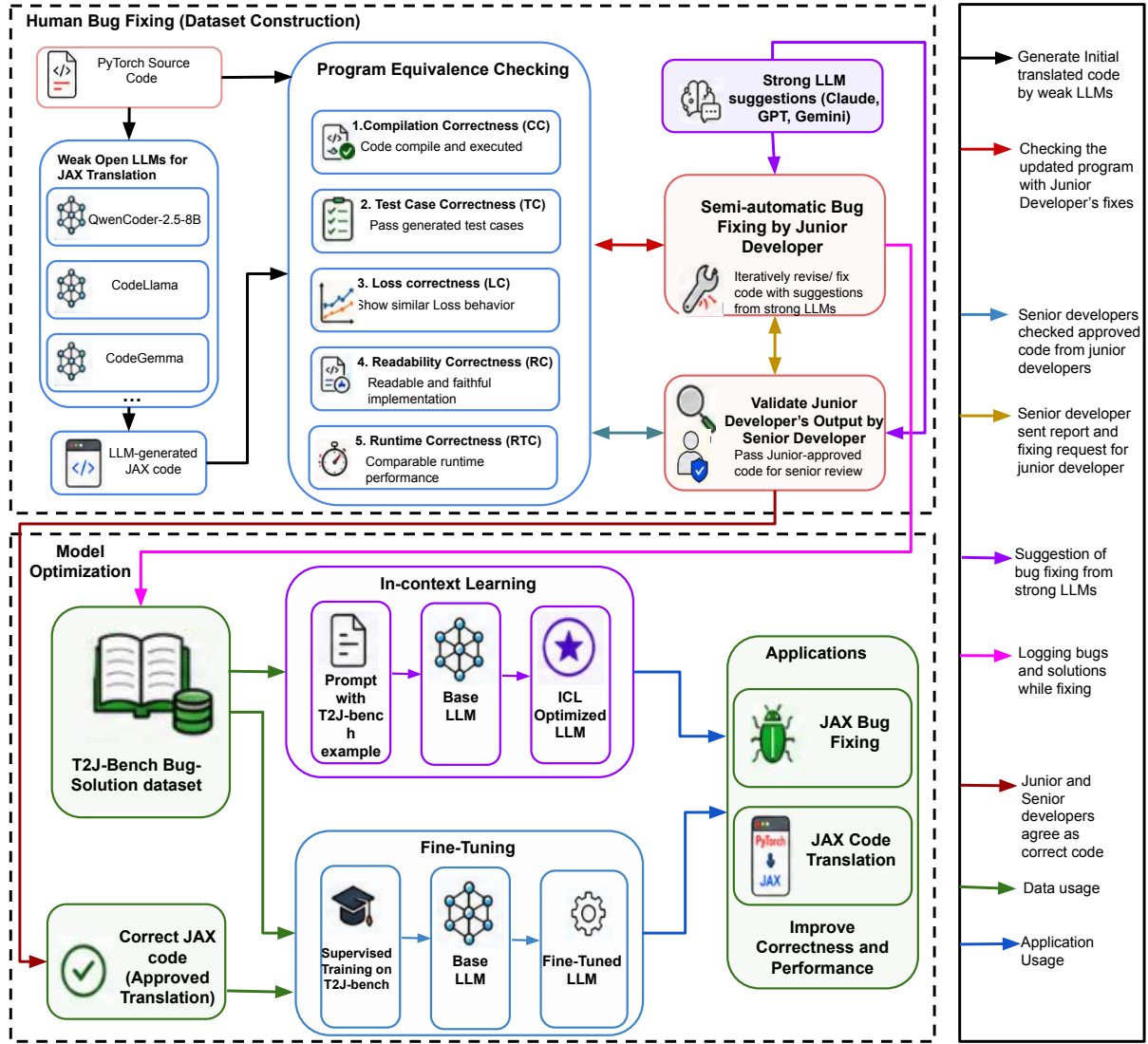


Figure 1: Overview of the Human Bug Fixing process for T2J-Bench and its applications with model optimization

3 Approach

3.1 Human Bug Fixing

The overview of the Human Bug Fixing process and its application in LLM Optimization was shown in Figure 1. Given an input PyTorch program and its LLM-generated JAX translation, we perform an iterative human bug-fixing process to construct verified JAX code and collect bug-solution pairs for T2J-Bench. First, junior developers conduct program equivalence checking between the PyTorch source code and the translated JAX code. If the JAX code fails any correctness criterion, the junior developers debug and revise the code, then repeat the equivalence checking process until the code passes their verification. The JAX code that passes this first stage is then forwarded to senior developers for an additional equivalence

check. If the senior developers identify remaining mistakes, the code is returned to the junior developers for further fixing and rechecking. This loop continues until both junior and senior developers agree that the final JAX implementation is equivalent to the input PyTorch program. The final verified JAX code is stored as the correct translation, while the identified bugs and their fixes are recorded as bug-solution examples in T2J-Bench.

3.2 Models Optimization

3.2.1 In-context Learning.

In the in-context learning setting, we use T2J-Bench as an external context resource without updating the parameters of the base LLM. The input to the model contains two parts: a standard prompt for PyTorch-to-JAX code translation and a context file that summarizes common error-solution tem-

plates observed during human bug fixing. These templates describe frequent mistakes in translating PyTorch code to JAX. In the prompt, we explicitly instruct the model to avoid or fix the error patterns described in the context file when generating the translated JAX code. This allows the model to use T2J-Bench as guidance at inference time and produce translations that are less likely to repeat common PyTorch-to-JAX translation errors.

3.2.2 Fine-tuning.

In the fine-tuning setting, we use T2J-Bench as supervised training data to directly optimize the base LLM. During the human bug-fixing process, each intermediate fixed JAX version and the final approved JAX code can be treated as a target label for training. Given the PyTorch source code and, when available, the buggy JAX translation as input, the model is trained to generate the corrected JAX implementation. Compared with in-context learning, fine-tuning internalizes the bug-fixing knowledge into the model parameters, enabling the fine-tuned model to generate more reliable JAX code without requiring all error-solution templates to be included in the prompt.

3.2.3 Application 1: Bug fixing

The first application of T2J-Bench is JAX bug fixing. The input consists of the original PyTorch code and a buggy JAX translation generated by an LLM. Instead of directly asking the model to translate from scratch, we use T2J-Bench as supervision for optimizing LLMs through in-context learning or fine-tuning. The optimized model learns common PyTorch-to-JAX translation errors and their corresponding fixes, then produces a fixed JAX version. The fixed code is finally evaluated by human verifiers to determine whether it satisfies the required program equivalence criteria. This application shows that T2J-Bench can be used as a repair-oriented resource for improving incorrect JAX translations.

3.2.4 Application 2: Code Translation

In this scenario, the optimized LLM receives PyTorch source code and translates it into JAX code. Compared with the bug-fixing setting, the model may also use an original JAX output from a previous LLM as additional context, but the final goal is to generate a correct JAX implementation. By optimizing LLMs with T2J-Bench through in-context learning or fine-tuning, the model can better avoid

common translation mistakes and produce JAX code that is more likely to pass human evaluation. This application demonstrates the usefulness of T2J-Bench not only for repairing buggy translations but also for improving the full translation pipeline.

4 Experiments

In this section, we evaluate the effectiveness of T2J-Bench for two downstream applications: JAX bug fixing and end-to-end PyTorch-to-JAX code translation. The goal of the experiments is to measure whether bug-solution examples collected from human debugging can improve the ability of LLMs to repair incorrect JAX code and generate correct JAX translations from PyTorch programs.

4.1 Selected Dataset

We construct T2J-Bench from the 20 kernels of the TorchLeet dataset. Each kernel contains a PyTorch implementation and corresponding test logic, which allows us to evaluate whether the translated JAX code preserves the behavior of the original PyTorch program. Starting from LLM-generated JAX translations, human developers perform iterative program equivalence checking and bug fixing. Through this process, we collect 600 bug-solution data points. Each data point records a buggy JAX code fragment, the corresponding fix, and the description of the error. These bug-solution examples are then used as supervision for in-context learning and fine-tuning.

Let $\mathcal{D}_{bug}$ denote the T2J-Bench bug-fixing dataset:

$$\mathcal{D}_{bug} = \{(P_i, J_i^{bug}, J_i^{fix}, e_i)\}_{i=1}^{600}, \quad (10)$$

where P_i is the PyTorch source code, J_i^{bug} is the buggy JAX translation, J_i^{fix} is the human-corrected JAX code, and e_i is the textual bug description or fixing explanation. For evaluation, we use the original 20 TorchLeet kernels as the test set and report the percentage of snippets that satisfy our program equivalence criteria.

4.2 Selected LLMs

We use QwenCoder-2.5-8B as the base LLM in our experiments. We compare three settings. The first setting is the original baseline, where the model is directly prompted to either fix buggy JAX code or translate PyTorch code into JAX without using

Table 1: Bug fixing results on 20 TorchLeet snippets.

Method	# Fixed / 20	Accuracy (%)
Original Baseline	8 / 20	40%
In-Context Learning	12 / 20	60%
Fine-Tuning	16 / 20	80%

T2J-Bench. The second setting is in-context learning, where relevant bug–solution examples from T2J-Bench are inserted into the prompt as demonstrations. The third setting is fine-tuning, where QwenCoder-2.5-8B is optimized on the 600 bug–solution pairs in T2J-Bench.

Formally, let M_{base} denote the original model, M_{icl} denote the model with T2J-Bench examples used as in-context demonstrations, and M_{ft} denote the model fine-tuned on T2J-Bench:

$$M_{base} = \text{QwenCoder-2.5-8B}, \quad (11)$$

$$M_{icl} = \text{ICL}(M_{base}, \mathcal{D}_{bug}), \quad (12)$$

$$M_{ft} = \text{FineTune}(M_{base}, \mathcal{D}_{bug}). \quad (13)$$

4.3 Evaluation Metric

We evaluate model performance using accuracy over the 20 TorchLeet snippets. A generated JAX program is considered correct only if it satisfies the five program equivalence criteria defined in Section 2: Compilation Correctness, Testcase Correctness, Loss Message Correctness, Readability Correctness, and Running Time Correctness. We evaluate the accuracy of bug fixing and code translation as the ratio of code snippets that can be translated to a correct JAX code in our evaluation dataset.

4.4 Results

4.4.1 Bug Fixing Results

Table 1 reports the results for the JAX bug-fixing application. In this setting, the model receives both the PyTorch source code and a buggy JAX translation as input. The goal is to generate a fixed JAX version that passes human equivalence checking. Compared with the original baseline, both T2J-Bench-based optimization strategies improve the percentage of fixed snippets. In-context learning improves the model by providing concrete examples of common PyTorch-to-JAX errors, while fine-tuning provides the strongest improvement because the model directly learns bug-fixing patterns from the 600 human-curated bug–solution pairs.

Table 2: Code translation results on 20 TorchLeet snippets. Accuracy is the percentage of PyTorch snippets that are translated into correct JAX code and accepted by program equivalence checking.

Method	# Correct / 20	Accuracy (%)
Original Baseline	7 / 20	35%
In-Context Learning with T2J-Bench	11 / 20	55%
Fine-Tuning with T2J-Bench	15 / 20	75%

4.4.2 Code Translation Results

Table 2 reports the results for the end-to-end PyTorch-to-JAX code translation application. In this setting, the model receives only the PyTorch source code as input and directly generates JAX code. The generated code is then evaluated by human verifiers using the same program equivalence criteria. The results show whether T2J-Bench can improve translation quality beyond the repair setting. Similar to the bug-fixing experiment, in-context learning uses T2J-Bench as prompt-level guidance, while fine-tuning uses T2J-Bench as training data. The expected trend is that fine-tuning achieves the highest translation accuracy because the model internalizes recurring translation rules, JAX API conventions, and common fixes.

4.5 Discussion

Overall, these experiments evaluate whether T2J-Bench is useful for both repairing existing JAX translations and generating JAX code from scratch. The bug-fixing task directly measures whether the model can learn from human-written fixes, while the code-translation task measures whether the same knowledge can help the model avoid bugs during generation. If both in-context learning and fine-tuning outperform the original baseline, this suggests that T2J-Bench captures reusable translation knowledge rather than only isolated bug fixes.

5 Analysis

5.1 Analysis on Human Bug-Fixing Process

In our human bug-fixing process, developers are not restricted to fixing the translated JAX code manually. Instead, they are allowed to use strong LLMs, such as Claude, Gemini, and GPT, as supporting tools for debugging and repairing the generated code. However, the developers are still responsible for verifying the generated fixes.

To better understand the effect of different strong LLMs on the human bug-fixing process, we analyze the average number of fixing steps required to repair JAX code generated by QwenCoder-2.5 on

Table 3: Average number of fixing steps on 10 selected hard problems. The buggy JAX translations are generated by QwenCoder-2.5, and strong LLMs are used as assistant tools during human debugging.

Assistant LLM	Avg. Fixing Steps	Observation
Claude	6	Fewest fixing steps
Gemini	8	Moderate fixing effort
GPT	10	Highest fixing effort

Table 4: Logged translation errors by model across four difficulty levels.

Model	Basic	Easy	Medium	Hard	Total
QwenCoder-2.5	48	62	71	49	230
Mistral-7B	33	35	25	25	118
CodeGemma-7B-It	19	26	20	15	80
CodeLlama-13B	38	37	28	28	131
CodeLlama-7B-Q8_0	34	37	26	26	123

10 selected hard PyTorch problems. Given a set of hard problems $\mathcal{H}$, we define the average number of fixing steps for an assistant model M as:

$$\text{AvgFixSteps}(M) = \frac{1}{|\mathcal{H}|} \sum_{i=1}^{|\mathcal{H}|} \text{FixSteps}(P_i, J_i^{\text{bug}}, M) \quad (14)$$

where P_i is the input PyTorch code, J_i^{bug} is the buggy JAX code generated by QwenCoder-2.5, and $\text{FixSteps}(\cdot)$ denotes the number of steps needed until the JAX code satisfies program equivalence checking. As shown in Table 3, Claude requires the fewest fixing steps on average, with 6 steps per problem. Gemini requires 8 steps, while GPT requires 10 steps. This result suggests that Claude provides the most effective debugging assistance among the three LLMs in this setting.

5.2 Analysis on Errors Performed by Open LLMs

We further analyze the error patterns produced by different open LLMs in PyTorch-to-JAX translation. Table 4 summarizes the total number of logged errors across difficulty levels. Overall, QwenCoder-2.5 produces the largest number of logged errors, with 230 total errors across Basic, Easy, Medium, and Hard problems. CodeGemma-7B-It produces the fewest logged errors, with 80 total errors. However, the number of logged errors should not be interpreted alone as model quality, because some models may generate shorter or incomplete translations that contain fewer explicit errors but fail to implement the full functionality. Therefore, we focus on both the number of errors and the types of errors produced by each model.

For QwenCoder-2.5, the dominant error types

vary by difficulty level. On Basic problems, its major issues include imports and versioning errors, optimizer API mistakes, and randomness or data generation mistakes. On Easy problems, it frequently fails on data pipeline fidelity, imports, and training-loop reporting. On Medium problems, QwenCoder-2.5 produces many data pipeline errors, shape and indexing errors, and parameter/state handling errors. On Hard problems, its most common issues are shape/indexing mistakes, optimizer state errors, and Flax module API misuse. These results indicate that QwenCoder-2.5 often generates detailed JAX code, but it struggles with framework-specific API correctness and execution semantics.

For Mistral-7B, the most important errors are related to Flax module APIs, optimizer APIs, and loss or gradient semantics. In Basic problems, Mistral-7B frequently makes optimizer API and Flax module API errors. In Easy and Medium problems, Flax module misuse becomes more dominant, especially in translating model classes and training components. On Hard problems, Mistral-7B has many loss and gradient semantic errors, indicating difficulty in preserving PyTorch behavior when custom gradients, GANs, sequence models, or explainability models are involved.

Overall, this analysis shows that open LLMs make different types of PyTorch-to-JAX translation errors. QwenCoder-2.5 tends to generate more detailed but error-prone translations involving APIs, shapes, and optimizer states. Mistral-7B and CodeGemma-7B-It frequently struggle with Flax and gradient semantics. CodeLlama-based models often fail to preserve data pipeline and task-level behavior. These observations motivate the need for T2J-Bench: by collecting human-verified bug-solution examples across multiple models and error categories, T2J-Bench provides targeted supervision for improving both JAX bug fixing and PyTorch-to-JAX code translation.

6 Related Works

6.1 Large Language Models for Code Translation

With the emergence of Large Language Models (LLMs), code translation has increasingly been addressed through prompting, instruction tuning, and LLM-based repair. Prior studies show that LLMs can perform translation among widely used programming languages, including C, C++, Java, and Python, but they still introduce semantic bugs, API

mismatches, and runtime failures (Yan et al., 2023; Pan et al., 2024). UniTrans proposes an iterative LLM-based correction framework that uses execution feedback and test information to improve translated programs (Yang et al., 2024). However, most existing studies focus on general-purpose languages, while domain-specific translation remains less explored. In contrast, our work focuses on PyTorch-to-JAX translation, where correctness depends not only on syntax and APIs but also on tensor semantics, gradient computation, loss behavior, random number generation, and runtime performance.

PyTorch-to-JAX translation is especially challenging because the two frameworks follow different programming and execution models. PyTorch commonly uses dynamic computation graphs and object-oriented model definitions, while JAX emphasizes functional transformations, explicit pseudo-random number generation, just-in-time compilation, and composable transformations such as `jit`, `grad`, and `vmap` (Bradbury et al., 2018). The original T2J work made an important step in this direction by collecting human-fixed PyTorch-to-JAX translation bugs and using them as context for improving LLM translation (Authors, 2025). Our work extends this direction by constructing T2J-Bench, a larger benchmark with 600 bug-solution examples, a clearer program equivalence definition, and two applications: JAX bug fixing and end-to-end PyTorch-to-JAX code translation.

6.2 Metrics for Code Evaluation

Evaluating translated code is difficult because textual similarity does not always reflect semantic correctness. Traditional metrics such as BLEU were originally designed for natural language generation and have been adopted in code generation and translation tasks, but prior work shows that BLEU-style metrics can be insufficient for code migration because two programs may be textually different while remaining functionally equivalent (Tran et al., 2019). CodeBLEU improves over BLEU by incorporating lexical similarity, abstract syntax tree matching, data-flow information, and weighted n-gram matching, making it more suitable for code synthesis and translation evaluation (Ren et al., 2020). Other representation-based metrics, such as CodeBERTScore, use pretrained code models to compare generated code with reference code in a semantic embedding space (Zhou et al., 2023).

Recent studies have also explored LLM-as-a-

judge methods for evaluating code quality. ICE-Score proposes using instruction-following LLMs to evaluate generated code along criteria such as functional correctness and usefulness (Zhuo, 2024). CodeUltraFeedback further studies LLM-based coding preference data for aligning models with code-quality judgments (Weyssow et al., 2024). The original T2J work adopted related ideas by defining LLM-based metrics for PyTorch-to-JAX translation, including usefulness, functional correctness, comparison score, and fixing cost (Authors, 2025). Our work differs by defining a stricter program equivalence framework that combines 5 well-known aspects in code correctness.

7 Limitations

The main limitation of this work is that the program equivalence checking process still relies heavily on human developers. Although human verification provides high-quality judgments, it is expensive, time-consuming, and difficult to scale to larger datasets or more complex machine learning programs. In particular, checking whether a translated JAX program correctly preserves training behavior, loss computation, and runtime performance often requires expert knowledge of both PyTorch and JAX. This human dependency limits the speed of dataset construction and may introduce subjective differences across verifiers.

8 Conclusion

In this paper, we introduced **T2J-Bench**, a benchmark and optimization resource for improving LLM-based PyTorch-to-JAX code translation. Unlike prior work that mainly evaluates translated code through limited human checking or surface-level similarity metrics, T2J-Bench focuses on a multi-dimensional definition of program equivalence, including compilation correctness, testcase correctness, loss message correctness, readability correctness, and running time correctness. By collecting 600 human-verified bug-solution examples from TorchLeet kernels, our work provides a larger and more systematic dataset for studying common translation failures in JAX code generated by LLMs. We further demonstrate that T2J-Bench can support two applications: JAX bug fixing and end-to-end PyTorch-to-JAX translation through in-context learning and fine-tuning.

References

- Anonymous Authors. 2025. Learning bug context for pytorch-to-jax translation with llms. Workshop paper.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. JAX: Composable transformations of python+numpy programs. <https://github.com/google/jax>.
- Zhihao Gong, Zeyu Sun, Dong Huang, Qingyuan Liang, Jie M. Zhang, and Dan Hao. 2025. TRACY: Benchmarking execution efficiency of llm-based code translation. *arXiv preprint arXiv:2508.11468*.
- Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, and 3 others. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.
- Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. *Lost in translation: A study of bugs introduced by large language models while translating code*. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*. Association for Computing Machinery.
- Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: A method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*.
- Ngoc Tran, Hieu Tran, Son Nguyen, Hoan Anh Nguyen, and Tien N. Nguyen. 2019. *Does BLEU score work for code migration?* In *Proceedings of the 27th International Conference on Program Comprehension, ICPC '19*, pages 165–176. IEEE.
- Martin Weyssow, Aton Kamanda, Xin Zhou, and Houari Sahraoui. 2024. CodeUltraFeedback: An llm-as-a-judge dataset for aligning large language models to coding preferences. *arXiv preprint arXiv:2403.09032*.
- Weixiang Yan, Yifei Tian, Yunzhe Li, Qian Chen, and Wen Wang. 2023. CodeTransOcean: A comprehensive multilingual benchmark for code translation. *arXiv preprint arXiv:2310.04951*.
- Zhen Yang, Fang Liu, Zhou Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yuting Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. *Exploring and unleashing the power of large language models in automated code translation*. *Proceedings of the ACM on Software Engineering, 1(FSE)*.
- Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. 2023. CodeBERTScore: Evaluating code generation with pretrained models of code. *arXiv preprint arXiv:2302.05527*.
- Terry Yue Zhuo. 2024. ICE-score: Instructing large language models to evaluate code. In *Findings of the Association for Computational Linguistics: EACL 2024*, pages 2232–2242. Association for Computational Linguistics.